

Module 9

Exploratory Data Analysis (EDA)

Complete Reference Guide — 7 Topics, 35 Subtopics

Dataset reference: `loan_prediction_db` | Tools: pandas, pure Python, matplotlib, seaborn

Prepared as a personal ML/Data Science exam-prep companion

Table of Contents

Topic 1	Understand EDA	1a–1e
Topic 2	Understand Descriptive Analysis	2a–2e
Topic 3	Understand Univariate Analysis	3a–3e
Topic 4	Understand Bivariate Analysis	4a–4e
Topic 5	Understand Multivariate Analysis	5a–5e
Topic 6	Understand Visualization Principles	6a–6e
Topic 7	Understand ML-Oriented EDA	7a–7e

Topic 1: Understand EDA

EDA means exploring your data carefully to understand it before doing any analysis or modeling.

1a. Purpose of EDA

Plain-English explanation:

EDA is the process of looking at your data before you do anything serious with it - cleaning, modeling, or drawing conclusions. It builds intuition about the data so later decisions are informed rather than guessed.

Real-world example:

Before a doctor prescribes treatment, they check vitals and symptoms first. EDA plays the same role for data - checking its 'vitals' before building anything on top of it.

ML / practical example (loan_prediction_db):

Before building a classifier on loan_prediction_db, check: How many rows/columns? Balance of defaulters vs non-defaulters? Missing values in income/credit score? Obvious outliers like income of 0 or age 150?

Code — pandas:

```
import pandas as pd

df = pd.read_csv('loan_data.csv')
df.shape
df.info()
df.describe()
```

Code — pure Python:

```
import csv
with open('loan_data.csv') as f:
    reader = csv.reader(f)
    header = next(reader)
    rows = list(reader)
print('Rows:', len(rows), 'Cols:', len(header))
```

Types / variations:

- Graphical EDA - visual (histograms, scatter plots)
- Non-graphical EDA - numerical (summary stats)
- Univariate / Bivariate / Multivariate approaches

Practice questions:

- Why is EDA a distinct step from data cleaning, even though both happen early?
- Write the minimum commands to explore a new DataFrame in the first 2 minutes.

Sources:

- <https://www.itl.nist.gov/div898/handbook/eda/eda.htm> - NIST EDA handbook

1b. Understanding datasets

Plain-English explanation:

Getting familiar with the basic shape and identity of a dataset - rows, columns, data types, and how columns relate conceptually.

Real-world example:

When moving into a new house, you walk through every room first before decorating - the same walkthrough applies to data.

ML / practical example (loan_prediction_db):

For loan_prediction_db: how many applicants (rows)? What columns exist - income, credit score, employment status? Which are numeric vs categorical vs target?

Code — pandas:

```
df.shape
df.columns
df.dtypes
df.head()
df.info()
```

Code — pure Python:

```
import csv
with open('loan_data.csv') as f:
    reader = csv.reader(f)
    header = next(reader)
    rows = list(reader)
print('Columns:', header)
```

Types / variations:

- Structured (CSV, SQL tables)
- Semi-structured (JSON)
- Unstructured (text, images)
- Numeric / categorical / datetime / text columns

Practice questions:

- Why must you know column data types before analyzing patterns?
- Print only numeric column names, then only categorical column names.

Sources:

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.info.html> - pandas info() docs
-

1c. Identifying patterns

Plain-English explanation:

Looking for repeating trends, relationships, or structures that show up consistently in the data - something that isn't random.

Real-world example:

A shopkeeper notices ice cream sales rise whenever temperature crosses 30C - a consistent, repeating pattern.

ML / practical example (loan_prediction_db):

In loan_prediction_db, applicants with lower income and high existing loans may default more often - a pattern connecting features to the target.

Code — pandas:

```
df.corr(numeric_only=True)
df.groupby('employment_type')['default'].mean()
```

Code — pure Python:

```
from collections import defaultdict
totals = defaultdict(float); counts = defaultdict(int)
for row in rows:
    totals[row['default']] += float(row['income'])
    counts[row['default']] += 1
```

Types / variations:

- Trend patterns
- Correlation patterns
- Cluster patterns
- Seasonal/cyclical patterns
- Association patterns

Practice questions:

- Is every correlation a pattern? Is every pattern a correlation?
- Find average loan_amount grouped by default status.

Sources:

- <https://www.itl.nist.gov/div898/handbook/eda/section3/eda33.htm> - NIST on data patterns
-

1d. Detecting anomalies

Plain-English explanation:

Actively looking for data points that don't fit the pattern - unusually high, low, or structurally wrong values.

Real-world example:

An attendance record showing 400 days attended in a month is obviously an error, not a real event.

ML / practical example (loan_prediction_db):

In loan_prediction_db, income = 0 or age = 150 are anomalies - likely placeholders or entry mistakes that could skew averages and model training.

Code — pandas:

```
Q1 = df['income'].quantile(0.25)
Q3 = df['income'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['income'] < Q1-1.5*IQR) | (df['income'] > Q3+1.5*IQR)]
```

Code — pure Python:

```
import statistics
mean = statistics.mean(incomes)
stdev = statistics.stdev(incomes)
anomalies = [x for x in incomes if abs(x-mean) > 3*stdev]
```

Types / variations:

- Point anomalies
- Contextual anomalies
- Collective anomalies
- IQR / Z-score / rule-based detection methods

Practice questions:

- What's the difference between an anomaly and a rare-but-valid data point?
- Detect anomalies in loan_amount using the IQR method.

Sources:

- <https://www.itl.nist.gov/div898/handbook/prc/section1/prc16.htm> - outlier detection reference
-

1e. Data storytelling

Plain-English explanation:

Taking discoveries (patterns, anomalies, distributions) and turning them into a clear narrative a human can understand and act on.

Real-world example:

A weather reporter doesn't show a raw temperature table - they say 'it's hot, carry water' - translating data into something actionable.

ML / practical example (loan_prediction_db):

Instead of a correlation table: 'Applicants earning under 20K/month with loans above 5L default nearly 3x more - suggesting income/debt should be strong model features.'

Code — pandas:

```
default_rate = df['default'].mean()*100
avg_income_def = df[df['default']==1]['income'].mean()
avg_income_non = df[df['default']==0]['income'].mean()
```

Code — pure Python:

```
defaulters = [r for r in rows if r['default']=='1']
non_defaulters = [r for r in rows if r['default']=='0']
avg_def = sum(float(r['income']) for r in defaulters)/len(defaulters)
```

Types / variations:

- Exploratory storytelling (internal notes)
- Explanatory storytelling (for stakeholders)
- Narrative + visual pairing

Practice questions:

- Why is a chart alone not considered good data storytelling?
- Compute average loan_amount by employment_type as an opening story sentence.

Sources:

- <https://www.storytellingwithdata.com/> - data storytelling principles
-

Topic 2: Understand Descriptive Analysis

Descriptive analysis means summarizing data using numbers like average, spread, and shape to describe it briefly.

2a. Summary statistics

Plain-English explanation:

A small set of numbers - average, min, max, spread - that describe the overall shape of a column at a glance.

Real-world example:

Describing a cricket player's season with average, highest score, and strike rate instead of every ball faced.

ML / practical example (loan_prediction_db):

Checking average, min, max, and std dev of income in loan_prediction_db shows if incomes are tightly clustered or widely spread.

Code — pandas:

```
df['income'].describe()
df.describe()
```

Code — pure Python:

```
import statistics
print(statistics.mean(incomes), min(incomes), max(incomes), statistics.stdev(incomes))
```

Types / variations:

- Central tendency (mean, median, mode)
- Spread (variance, std dev, IQR)
- Position (min, max, percentiles)
- Count-based (count, unique, missing)

Practice questions:

- Why is std dev as important as the mean?
- Print summary stats for only loan_amount and income.

Sources:

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html> - describe() docs

2b. Distribution analysis

Plain-English explanation:

Looking at how values spread across a range - clustered, skewed, or even - to see the full shape of a variable.

Real-world example:

Exam scores often follow a bell shape; household income usually skews right, with a few high earners pulling the tail.

ML / practical example (loan_prediction_db):

If income is heavily right-skewed, a linear model may need a log transform to make the distribution more symmetric.

Code — pandas:

```
df['income'].skew()
df['income'].kurt()
df['income'].value_counts(bins=5)
```

Code — pure Python:

```
mean, median = statistics.mean(incomes), statistics.median(incomes)
if mean > median: print('right-skewed')
```

Types / variations:

- Normal (symmetric)
- Right-skewed
- Left-skewed
- Uniform
- Bimodal/multimodal

Practice questions:

- If mean > median, what does that say about distribution shape?
- Check skewness of income and loan_amount, print which is more skewed.

Sources:

- <https://www.analyticsvidhya.com/blog/2021/05/skewness-and-kurtosis/> - skewness/kurtosis guide
-

2c. Variability analysis

Plain-English explanation:

Measuring how spread out or consistent values are - two datasets can share a mean but differ greatly in spread.

Real-world example:

Two delivery riders both average 30 minutes, but one is consistently 28-32 min while the other swings 10-60 min.

ML / practical example (loan_prediction_db):

High variability in loan_amount (small personal loans to huge business loans) may require segmenting applicants or careful scaling.

Code — pandas:

```
df['loan_amount'].var()
df['loan_amount'].std()
cv = df['loan_amount'].std()/df['loan_amount'].mean()
```

Code — pure Python:

```
variance = statistics.variance(amounts)
std_dev = statistics.stdev(amounts)
value_range = max(amounts)-min(amounts)
```

Types / variations:

- Range
- Variance
- Standard deviation
- Coefficient of variation (CV)
- Interquartile range (IQR)

Practice questions:

- Why prefer IQR over std dev when extreme outliers exist?
- Compute CV for income and loan_amount, compare variability.

Sources:

- <https://www.itl.nist.gov/div898/handbook/eda/section3/eda356.htm> - measures of spread

2d. Central tendency analysis

Plain-English explanation:

Finding the single 'typical' value of a dataset - mean, median, or mode - each telling a slightly different story.

Real-world example:

A 'typical salary' via mean can be skewed by a CEO's pay; median gives a more realistic picture of most employees.

ML / practical example (loan_prediction_db):

For filling missing income values, median is often preferred over mean when income is skewed by a few high earners.

Code — pandas:

```
df['income'].mean()  
df['income'].median()  
df['income'].mode()[0]
```

Code — pure Python:

```
print(statistics.mean(incomes), statistics.median(incomes), statistics.mode(incomes))
```

Types / variations:

- Mean (symmetric, no outliers)
- Median (skewed data/outliers)
- Mode (categorical data)
- Weighted mean

Practice questions:

- For a categorical column, which central tendency measure applies and why not the others?
- Compute mean/median/mode of loan_amount and judge symmetry from the gap.

Sources:

- <https://www.scribbr.com/statistics/central-tendency/> - central tendency guide
-

2e. Dataset profiling

Plain-English explanation:

Generating a complete structured overview of a dataset in one go - combining types, missing values, stats, and distributions into a single report.

Real-world example:

Like a full medical check-up report bundling blood pressure, sugar, and weight into one document instead of separate checks.

ML / practical example (loan_prediction_db):

Before working on loan_prediction_db, generate a profile showing missing % per column, dtypes, unique counts, distributions, and correlations.

Code — pandas:

```
profile = pd.DataFrame({'dtype': df.dtypes, 'missing_pct': df.isnull().mean()*100, 'unique': df.nunique()})
```

Code — pure Python:

```
for col in header:  
    values = [row[col] for row in rows]
```

```
missing = sum(1 for v in values if v=='')  
print(col, missing, len(set(values)))
```

Types / variations:

- Manual profiling
- Automated tools (ydata-profiling, Sweetviz)
- Column-level vs dataset-level profiling

Practice questions:

- Why is profiling done before modeling starts rather than midway?
- Build a profiling table sorted by missing percentage descending.

Sources:

- <https://docs.profiling.ydata.ai/> - ydata-profiling documentation
-

Topic 3: Understand Univariate Analysis

Univariate analysis means studying just one variable at a time, on its own, without comparing it to any other variable.

3a. Single-variable analysis

Plain-English explanation:

Examining one column in isolation - its values, spread, and range - without relating it to other columns yet.

Real-world example:

A teacher first looks at one student's marks across subjects before comparing to other students.

ML / practical example (loan_prediction_db):

Before checking if income relates to default, study income alone: range, typical value, missing values.

Code — pandas:

```
income = df['income']
income.describe()
income.isnull().sum()
```

Code — pure Python:

```
incomes = [float(row['income']) for row in reader if row['income']]
print(min(incomes), max(incomes), len(incomes))
```

Types / variations:

- Numeric single-variable analysis (stats, histograms)
- Categorical single-variable analysis (frequency counts)

Practice questions:

- Why understand a variable individually before relating it to others?
- Isolate the age column and print min, max, mean.

Sources:

- <https://www.statology.org/univariate-analysis/> - univariate analysis fundamentals

3b. Histograms

Plain-English explanation:

A bar-chart-like visual showing how many data points fall into value ranges (bins) - the primary way to see a variable's distribution shape.

Real-world example:

Grouping park visitors by height range and counting how many fall in each range, then drawing bars.

ML / practical example (loan_prediction_db):

Plotting a histogram of loan_amount shows if most loans are small with a few very large ones (right-skewed).

Code — pandas:

```
df['loan_amount'].hist(bins=20)
plt.xlabel('Loan Amount'); plt.show()
```

Code — pure Python:

```
bin_size = 50000
bins = {}
for amt in amounts:
    key = int(amt // bin_size) * bin_size
```

```
bins[key] = bins.get(key, 0) + 1
```

Types / variations:

- Equal-width bins
- Equal-frequency bins
- Density histogram

Practice questions:

- What happens if you choose too few vs too many bins?
- Plot a histogram of age with 15 bins using matplotlib.

Sources:

- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html - matplotlib hist docs
-

3c. Frequency distributions

Plain-English explanation:

A table or count showing how often each distinct value or category appears in a column - the numeric version of a histogram for categories.

Real-world example:

Counting how many students got grade A, B, C, D in a class and listing counts.

ML / practical example (loan_prediction_db):

A frequency distribution of employment_type tells you class balance - useful since imbalanced categories can bias a model.

Code — pandas:

```
df['employment_type'].value_counts()  
df['employment_type'].value_counts(normalize=True)*100
```

Code — pure Python:

```
from collections import Counter  
freq = Counter(types)  
for cat, count in freq.items(): print(cat, count)
```

Types / variations:

- Absolute frequency
- Relative frequency
- Cumulative frequency

Practice questions:

- When is relative frequency more useful than absolute counts?
- Show frequency distribution of default as percentages.

Sources:

- https://pandas.pydata.org/docs/reference/api/pandas.Series.value_counts.html - value_counts docs
-

3d. Outlier detection

Plain-English explanation:

Finding values in a single column that lie far outside its normal range, using box plots, IQR, or Z-score - closely related to anomaly detection but applied one variable at a time.

Real-world example:

In a list of salaries 30-60K, one entry of 5 crore is a visible outlier on a box plot.

ML / practical example (loan_prediction_db):

Box-plotting income in loan_prediction_db may reveal extreme earners worth checking before training a model.

Code — pandas:

```
import seaborn as sns
sns.boxplot(x=df['income'])
Q1, Q3 = df['income'].quantile([0.25, 0.75])
```

Code — pure Python:

```
incomes_sorted = sorted(incomes)
n = len(incomes_sorted)
Q1, Q3 = incomes_sorted[n//4], incomes_sorted[3*n//4]
```

Types / variations:

- Box plot method
- IQR method
- Z-score method

Practice questions:

- How does univariate outlier detection differ from anomaly detection in 1d?
- Create a seaborn box plot for loan_amount.

Sources:

- <https://seaborn.pydata.org/generated/seaborn.boxplot.html> - seaborn boxplot docs
-

3e. Distribution interpretation

Plain-English explanation:

Looking at histogram shape, skewness, and outliers together for one variable to draw a conclusion about what it means for the analysis or model.

Real-world example:

A doctor doesn't just show blood test numbers - they interpret them: 'cholesterol is high, keep an eye on it.'

ML / practical example (loan_prediction_db):

Conclude: 'Income is right-skewed with outliers; recommend log transform and median imputation before modeling.'

Code — pandas:

```
skew = df['income'].skew()
if skew > 1: print('Highly right-skewed - consider log transform')
```

Code — pure Python:

```
gap = statistics.mean(incomes) - statistics.median(incomes)
print('Right-skewed' if gap > 0 else 'Left-skewed or symmetric')
```

Types / variations:

- Interpretation for modeling
- Interpretation for business
- Interpretation for data quality

Practice questions:

- Why is interpretation the most important step of univariate analysis?

- Check skewness of loan_amount and recommend a transform if needed.

Sources:

- <https://www.analyticsvidhya.com/blog/2021/05/skewness-and-kurtosis/> - skewness interpretation guide
-

Topic 4: Understand Bivariate Analysis

Bivariate analysis means studying the relationship between exactly two variables at a time.

4a. Feature relationships

Plain-English explanation:

Examining how two variables move together or connect - does one variable's value influence, predict, or associate with another's?

Real-world example:

A teacher notices students who study more hours tend to score higher marks - a relationship between two variables.

ML / practical example (loan_prediction_db):

Check the relationship between income and loan_amount - do higher earners request bigger loans?

Code — pandas:

```
df.groupby('employment_type')['loan_amount'].mean()  
pd.crosstab(df['employment_type'], df['default'])
```

Code — pure Python:

```
from collections import defaultdict  
totals = defaultdict(float); counts = defaultdict(int)  
for row in rows:  
    totals[row['employment_type']] += float(row['loan_amount'])
```

Types / variations:

- Numeric-numeric (correlation, scatter)
- Categorical-categorical (cross-tabs)
- Numeric-categorical (group-by, box plots)

Practice questions:

- Why check feature relationships before feature selection?
- Check average income grouped by default status.

Sources:

- <https://pandas.pydata.org/docs/reference/api/pandas.crosstab.html> - pandas crosstab docs

4b. Correlation analysis

Plain-English explanation:

Measuring the strength and direction of a relationship between two numeric variables using one number from -1 to +1.

Real-world example:

Height and weight are usually positively correlated - taller people tend to weigh more, though not always.

ML / practical example (loan_prediction_db):

If income and loan_amount correlate at 0.7, that's a strong positive relationship worth noting for multicollinearity.

Code — pandas:

```
df['income'].corr(df['loan_amount'])  
df.corr(numeric_only=True)
```

Code — pure Python:

```
numerator = sum((incomes[i]-mean_i)*(amounts[i]-mean_a) for i in range(n))
denom = (sum((x-mean_i)**2 for x in incomes)*sum((y-mean_a)**2 for y in amounts))**0.5
```

Types / variations:

- Pearson correlation (linear)
- Spearman correlation (rank-based/monotonic)
- Kendall correlation (rank-based)

Practice questions:

- Why doesn't a correlation of 0 always mean 'no relationship exists'?
- Compute correlation between age and loan_amount.

Sources:

- <https://www.scribbr.com/statistics/pearson-correlation-coefficient/> - Pearson correlation explained
-

4c. Scatter plots

Plain-English explanation:

A chart plotting each record as a point using two numeric variables - lets you see the actual shape of a relationship, not just a number.

Real-world example:

Plotting training hours (x) against race finish time (y) - a downward trend shows more training means faster times.

ML / practical example (loan_prediction_db):

A scatter plot of income vs loan_amount may reveal most points trending upward with a few outlier points breaking the pattern.

Code — pandas:

```
import seaborn as sns
sns.scatterplot(x='income', y='loan_amount', data=df)
```

Code — pure Python:

```
for row in rows[:10]:
    print(row['income'], row['loan_amount'])
```

Types / variations:

- Positive linear scatter
- Negative linear scatter
- No pattern scatter
- Non-linear scatter

Practice questions:

- Why might correlation near 0 still show a clear pattern on a scatter plot?
- Create a scatter plot of age vs income.

Sources:

- <https://seaborn.pydata.org/generated/seaborn.scatterplot.html> - seaborn scatterplot docs
-

4d. Comparative analysis

Plain-English explanation:

Comparing a numeric variable across groups defined by a categorical variable - does the number differ depending on category?

Real-world example:

Comparing average exam scores between students who attended extra classes vs those who didn't.

ML / practical example (loan_prediction_db):

Comparing average default rate between Salaried vs Self-employed applicants tells if employment type matters for risk.

Code — pandas:

```
df.groupby('employment_type')['default'].mean()  
sns.boxplot(x='employment_type', y='loan_amount', data=df)
```

Code — pure Python:

```
for row in rows:  
    totals[row['employment_type']] += float(row['default'])  
    counts[row['employment_type']] += 1
```

Types / variations:

- Group-by mean/median comparison
- Box plot comparison
- T-test/ANOVA (statistical significance testing)

Practice questions:

- Why is comparative analysis 'bivariate' even with one categorical variable?
- Compare average income across employment_type groups.

Sources:

- <https://seaborn.pydata.org/generated/seaborn.boxplot.html> - seaborn boxplot for group comparison
-

4e. Trend identification

Plain-English explanation:

Spotting a consistent directional pattern between two variables over an ordered sequence, most commonly over time.

Real-world example:

Tracking a company's monthly sales over a year - if sales consistently rise, that's an upward trend.

ML / practical example (loan_prediction_db):

Checking if default_rate has trended up or down over recent months could reveal changing economic conditions.

Code — pandas:

```
df['application_date'] = pd.to_datetime(df['application_date'])  
df.groupby(df['application_date'].dt.to_period('M'))['default'].mean()
```

Code — pure Python:

```
for row in rows:  
    month = row['application_date'][:7]  
    totals[month] += float(row['default'])
```

Types / variations:

- Upward trend
- Downward trend
- Stable/flat trend

- Seasonal trend

Practice questions:

- What's the risk of assuming a short-term trend continues indefinitely?
- Plot the trend of average loan_amount over months.

Sources:

- https://pandas.pydata.org/docs/reference/api/pandas.Series.dt.to_period.html - pandas period grouping
-

Topic 5: Understand Multivariate Analysis

Multivariate analysis means studying the relationships between three or more variables at the same time.

5a. Multiple feature relationships

Plain-English explanation:

Extends bivariate analysis to look at three or more variables together, seeing how they jointly relate - not just pairwise.

Real-world example:

A student's exam score is shaped by study hours, sleep, and stress together, not any one factor alone.

ML / practical example (loan_prediction_db):

Default risk in loan_prediction_db likely depends on income, loan_amount, and employment_type together.

Code — pandas:

```
df.groupby(['employment_type', 'default'])['income'].mean()
df.pivot_table(values='loan_amount', index='employment_type', columns='default', aggfunc='mean')
```

Code — pure Python:

```
for row in rows:
    key = (row['employment_type'], row['default'])
    totals[key] += float(row['income'])
```

Types / variations:

- Grouped aggregation
- Pivot tables
- Multiple regression (modeling technique)

Practice questions:

- Why can bivariate analysis give a misleading picture vs 3+ variables together?
- Create a pivot table of average income by employment_type and default.

Sources:

- https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.pivot_table.html - pivot table docs

5b. Feature interaction analysis

Plain-English explanation:

Checking whether the effect of one variable on an outcome changes depending on the value of another variable.

Real-world example:

A medicine might lower blood pressure in young patients but have little effect in older patients - age and medicine interact.

ML / practical example (loan_prediction_db):

High loan_amount might strongly increase default risk for Self-employed but barely matter for Salaried applicants.

Code — pandas:

```
import seaborn as sns
sns.lmplot(x='loan_amount', y='default', hue='employment_type', data=df, logistic=True)
```

Code — pure Python:

```
groups = defaultdict(lambda: defaultdict(list))
```

```
for row in rows:
    bucket = int(float(row['loan_amount'])//100000)*100000
    groups[row['employment_type']][bucket].append(float(row['default']))
```

Types / variations:

- Two-way interaction
- Three-way interaction
- Visual interaction check (separate trend lines)
- Statistical interaction term (regression)

Practice questions:

- How is an interaction different from two features simply both correlating with the outcome?
- Plot income vs default rate separated by employment_type.

Sources:

- <https://seaborn.pydata.org/generated/seaborn.lmplot.html> - seaborn lmplot docs

5c. Pattern discovery

Plain-English explanation:

Finding groupings, clusters, or combinations of feature values that repeatedly occur together across multiple variables.

Real-world example:

A supermarket discovers customers who buy diapers, shop late, and pay by card also tend to buy baby formula.

ML / practical example (loan_prediction_db):

A segment of Self-employed, low income, high loan_amount applicants might have a much higher default rate than any single factor suggests.

Code — pandas:

```
high_risk = df[(df['employment_type']=='Self-employed') & (df['income']<25000) & (df['loan_amount']>500000)]
high_risk['default'].mean()
```

Code — pure Python:

```
segment = [row for row in rows if row['employment_type']=='Self-employed' and float(row['income'])<25000 and float(row['loan_amount'])>500000]
```

Types / variations:

- Rule-based pattern discovery
- Clustering-based discovery (K-Means)
- Association rule mining

Practice questions:

- Why might a pattern only emerge with 3 variables combined, invisible in any 2-variable view?
- Define a different high-risk segment using age, income, loan_amount.

Sources:

- <https://scikit-learn.org/stable/modules/clustering.html> - clustering methods overview

5d. Dimensional thinking

Plain-English explanation:

Recognizing that each variable adds a new 'dimension' to the data - relationships invisible in 2D can become visible in 3D or higher.

Real-world example:

Judging a house's value by size alone is 1D; adding location, age, rooms makes it richer but harder to visualize directly.

ML / practical example (loan_prediction_db):

Default risk exists in a space defined by dozens of features - models like trees or neural nets navigate this multi-dimensional space.

Code — pandas:

```
sns.scatterplot(x='income', y='loan_amount', hue='employment_type', size='age', data=df)
```

Code — pure Python:

```
sample_point = rows[0]
for col, val in sample_point.items():
    print(col, val)
```

Types / variations:

- Low-dimensional thinking
- High-dimensional thinking
- Dimensionality reduction (PCA)

Practice questions:

- Why does adding more dimensions make a dataset harder to visualize but more accurate?
- Plot income vs age with hue as default status.

Sources:

- <https://seaborn.pydata.org/tutorial/relational.html> - encoding extra dimensions with hue/size
-

5e. Business insights generation

Plain-English explanation:

Taking multivariate findings and translating them into a clear, actionable recommendation for a business or stakeholder.

Real-world example:

An analyst tells a manager: 'customers who buy X and Y together in evenings are 40% more likely to buy Z - bundle these in evening promotions.'

ML / practical example (loan_prediction_db):

'Self-employed applicants with loans above 5L and income below 25K default at nearly 3x the overall rate - recommend stricter approval criteria.'

Code — pandas:

```
segment_rate = segment['default'].mean()
overall_rate = df['default'].mean()
print(segment_rate/overall_rate)
```

Code — pure Python:

```
overall_defaults = sum(1 for r in rows if r['default']=='1')/len(rows)
segment_defaults = sum(1 for r in segment if r['default']=='1')/len(segment)
```

Types / variations:

- Risk-segment insights

- Opportunity insights
- Operational insights
- Predictive insights

Practice questions:

- Why is a multivariate insight more convincing to stakeholders than a univariate one?
- Find another 3-variable high-risk segment and print an insight sentence.

Sources:

- <https://hbr.org/2018/11/if-you-want-people-to-listen-stop-talking> - business insight communication reference
-

Topic 6: Understand Visualization Principles

Visualization principles are the rules and best practices for choosing and designing charts so they communicate data honestly and clearly.

6a. Choosing the right chart

Plain-English explanation:

Picking the visualization type that matches the data and the question - comparison, distribution, relationship, or trend.

Real-world example:

A pie chart works for showing expense splits, but a line chart is needed to show expenses changing month by month.

ML / practical example (loan_prediction_db):

Use a histogram for income distribution, a scatter plot for income vs loan_amount, a bar chart for default rate by employment_type.

Code — pandas:

```
df['income'].hist()
sns.scatterplot(x='income', y='loan_amount', data=df)
df['employment_type'].value_counts().plot(kind='bar')
```

Code — pure Python:

```
def suggest_chart(num_vars, var_types, has_time_order):
    if num_vars == 1 and var_types[0]=='numeric': return 'Histogram'
    if num_vars == 2: return 'Scatter plot'
```

Types / variations:

- Distribution charts (histogram, box plot)
- Relationship charts (scatter, heatmap)
- Comparison charts (bar chart)
- Trend charts (line chart)

Practice questions:

- Why is a pie chart a poor choice for loan_amount distribution across 1000 applicants?
- Pick and plot the right chart for age vs default rate.

Sources:

- <https://www.data-to-viz.com/> - chart selection guide by data type

6b. Data storytelling (visual)

Plain-English explanation:

Designing a chart so it tells a clear story on its own - with the right title, labels, highlighting, and framing.

Real-world example:

A weather app highlights 'today' clearly and labels 'rain expected at 3pm' instead of just showing raw numbers.

ML / practical example (loan_prediction_db):

Highlight the high-risk cluster in red on a scatter plot with a title stating the insight directly, not just variable names.

Code — pandas:

```
ax.scatter(df['income'], df['loan_amount'], c='lightgray')
```

```
ax.scatter(high_risk['income'], high_risk['loan_amount'], c='red')
ax.set_title('High-Risk Applicants: Low Income, High Loan Amount')
```

Code — pure Python:

```
high_risk = [r for r in rows if float(r['income'])<25000 and float(r['loan_amount'])>500000]
print(f'{len(high_risk)} applicants fall into a high-risk cluster')
```

Types / variations:

- Title-driven storytelling
- Color-driven highlighting
- Annotation-driven storytelling

Practice questions:

- Why does a descriptive title communicate better than a generic one?
- Plot age vs default, highlighting under-25 applicants, with a descriptive title.

Sources:

- <https://www.storytellingwithdata.com/> - visual storytelling principles
-

6c. Visualization best practices

Plain-English explanation:

General rules that make any chart clearer and more trustworthy - proper labels, scales, readable colors, minimal clutter.

Real-world example:

A cluttered road sign is harder to read at a glance than a clean, simple one - even with the same information.

ML / practical example (loan_prediction_db):

A clean bar chart of 'Default Rate by Employment Type' with clear labels communicates better than an unlabeled rainbow 3D chart.

Code — pandas:

```
data = df.groupby('employment_type')['default'].mean().sort_values()
data.plot(kind='barh', color='#1D9E75')
ax.set_xlabel('Default Rate')
```

Code — pure Python:

```
def check_chart_practices(has_title, has_axis_labels, num_colors):
    issues = []
    if not has_title: issues.append('Missing title')
    return issues
```

Types / variations:

- Labeling practices (titles, axis labels)
- Color practices (limited, consistent palette)
- Scale practices (start bars at zero)
- Clutter reduction

Practice questions:

- Why is starting a bar chart's y-axis at non-zero a best-practice violation?
- Plot default rate by employment_type, sorted, with clear labels.

Sources:

- <https://matplotlib.org/stable/users/explain/customizing.html> - matplotlib customization docs
-

6d. Avoiding misleading visuals

Plain-English explanation:

Recognizing chart choices that distort perception - truncated axes, inconsistent scales, cherry-picked ranges, 3D distortion.

Real-world example:

A bar chart starting its y-axis at 90 instead of 0 can make a 2% difference look like a massive difference.

ML / practical example (loan_prediction_db):

A truncated y-axis (10%-15%) on default rate by employment_type could make a real 11% vs 13% difference look exaggerated.

Code — pandas:

```
axes[0].set_ylim(0.10, 0.15) # misleading
axes[1].set_ylim(0, 1) # honest
```

Code — pure Python:

```
def is_axis_misleading(y_min, y_max, data_min, data_max):
    axis_range = y_max - y_min
    data_range = data_max - data_min
    return axis_range < data_range * 3
```

Types / variations:

- Truncated/non-zero axis
- Inconsistent scales
- Cherry-picked ranges
- 3D/perspective distortion

Practice questions:

- Name one other way a chart can be technically correct but still mislead.
- Create a misleading chart then a corrected, honest version.

Sources:

- https://www.callingbullshit.org/tools/tools_misleading_axes.html - misleading axis techniques
-

6e. Effective communication

Plain-English explanation:

The overarching goal tying together chart choice, storytelling, best practices, and honesty - tailored to the audience.

Real-world example:

A pilot's cockpit shows the most critical info instantly visible - prioritizing what the audience needs to see.

ML / practical example (loan_prediction_db):

Technical teams get a detailed correlation heatmap; business stakeholders get one clean chart with a single headline insight.

Code — pandas:

```
sns.heatmap(df.corr(numeric_only=True), annot=True) # technical
plt.bar(['Overall', 'High-Risk'], [overall_rate, high_risk_rate]) # executive
```

Code — pure Python:

```
def tailor_message(audience, insight):
    if audience == 'executive': return f'Headline: {insight}'
```

```
return f'Detailed finding: {insight}'
```

Types / variations:

- Technical communication
- Executive communication
- General public communication

Practice questions:

- Why might a detailed heatmap reduce effectiveness for executives, despite being more complete?
- Generate a detailed and a simplified version of the same finding.

Sources:

- <https://www.tableau.com/learn/articles/data-visualization-tips> - visualization communication tips
-

Topic 7: Understand ML-Oriented EDA

ML-Oriented EDA means doing exploratory analysis specifically to prepare data for machine learning.

7a. Target variable analysis

Plain-English explanation:

Studying the column you're trying to predict - its distribution, balance, and relationship to other variables - before modeling.

Real-world example:

A teacher checks how students' scores are distributed before designing a grading curve.

ML / practical example (loan_prediction_db):

Check if default is binary, and its class balance - rare defaults (5%) vs common (40%) changes the modeling approach needed.

Code — pandas:

```
df['default'].value_counts()  
df['default'].value_counts(normalize=True)*100
```

Code — pure Python:

```
from collections import Counter  
counts = Counter(targets)  
for cls, count in counts.items(): print(cls, count/len(targets))
```

Types / variations:

- Binary target
- Multi-class target
- Continuous target (regression)
- Balanced vs imbalanced target

Practice questions:

- Why does a heavily imbalanced target make accuracy a misleading metric?
- Check if default is imbalanced and warn if a class is under 20%.

Sources:

- <https://machinelearningmastery.com/what-is-imbalanced-classification/> - imbalanced classification overview

7b. Feature importance awareness

Plain-English explanation:

Forming an early, data-driven sense of which features likely matter most for the target, before any model is trained.

Real-world example:

A coach already has a rough sense of the strongest players from practice, before an official match.

ML / practical example (loan_prediction_db):

Income and loan_amount show strong relationships with default from earlier EDA; applicant_id clearly wouldn't matter.

Code — pandas:

```
df.corr(numeric_only=True)['default'].sort_values(ascending=False)  
df.groupby('employment_type')['default'].mean()
```

Code — pure Python:

```
cov = sum((incomes[i]-mean_i)*(targets[i]-mean_t) for i in range(n))
correlation = cov/denom
```

Types / variations:

- Correlation-based estimate
- Group-difference estimate
- Formal feature importance (post-training)

Practice questions:

- Why is EDA-level feature importance a rough estimate, not a reliable ranking?
- Rank numeric features by absolute correlation with default.

Sources:

- https://scikit-learn.org/stable/modules/permutation_importance.html - formal feature importance reference
-

7c. Bias identification

Plain-English explanation:

Checking whether the dataset systematically over/under-represents groups or bakes in historical unfairness that could bias predictions.

Real-world example:

A hiring dataset with few women in senior roles due to past bias could teach a model to unfairly favor men again.

ML / practical example (loan_prediction_db):

If a region or employment type was historically rejected more for non-financial reasons, a model could repeat that unfair pattern.

Code — pandas:

```
df.groupby('employment_type')['default'].agg(['mean', 'count'])
df['employment_type'].value_counts(normalize=True)*100
```

Code — pure Python:

```
for group in group_totals:
    rate = group_defaults[group]/group_totals[group]
    share = group_totals[group]/len(rows)
```

Types / variations:

- Representation bias
- Historical/label bias
- Measurement bias
- Sampling bias

Practice questions:

- Why can a model be 'accurate' on paper but still unfair in practice?
- Check representation and default rate per group, flag groups under 5%.

Sources:

- <https://fairmlbook.org/> - fairness and bias in ML datasets
-

7d. Data quality validation

Plain-English explanation:

A final ML-focused check confirming the dataset is usable for modeling - no missing values, duplicates, or data leakage.

Real-world example:

A restaurant manager does a final walkthrough each morning checking ingredients and equipment before opening.

ML / practical example (loan_prediction_db):

Confirm no missing values remain in income/loan_amount, no duplicate applicants, and no column accidentally leaks the default outcome.

Code — pandas:

```
df.isnull().sum()
df.duplicated().sum()
df.corr(numeric_only=True)['default']
```

Code — pure Python:

```
seen = set()
duplicates = 0
for row in rows:
    key = tuple(row.values())
    if key in seen: duplicates += 1
    seen.add(key)
```

Types / variations:

- Completeness check
- Uniqueness check
- Consistency check
- Leakage check

Practice questions:

- What is data leakage and why is it dangerous despite inflating accuracy?
- Check for duplicate records and print how many exist.

Sources:

- <https://machinelearningmastery.com/data-leakage-machine-learning/> - data leakage explanation
-

7e. Preparing for model building

Plain-English explanation:

Consolidating every EDA finding into a final readiness checklist and plan before moving to actual model building.

Real-world example:

Before a road trip, you check fuel, route, and service - final prep informed by everything checked beforehand.

ML / practical example (loan_prediction_db):

Final handoff: 'Target imbalanced (15% default) - use stratified sampling and F1-score. Income right-skewed - apply log transform. No leakage detected.'

Code — pandas:

```
print(df['default'].mean())
print(df.isnull().sum().sum())
print(df.corr(numeric_only=True)['default'].drop('default').abs().idxmax())
```

Code — pure Python:

```
default_rate = sum(int(r['default']) for r in rows)/len(rows)
missing = sum(1 for r in rows for v in r.values() if v=='')
```

Types / variations:

- Preprocessing checklist
- Modeling approach checklist
- Feature checklist

Practice questions:

- Why document EDA findings as a checklist rather than just remembering them?
- Print a readiness summary: target balance, missing values, duplicate count.

Sources:

- <https://developers.google.com/machine-learning/data-prep> - Google ML data preparation guide
-

Rapid Recap — All Topics (Rule 17 Format)

One combined sentence per subtopic: meaning + difference + interview-ready phrasing.

Recap: Topic 1: Understand EDA

EDA means exploring your data carefully to understand it before doing any analysis or modeling.

1a. Purpose of EDA: EDA is the process of understanding data before using it - unlike cleaning, which fixes data, EDA's job is only to understand it - as you'd say: 'EDA is the process of understanding data before using it, so decisions are based on facts, not guesses.'

1b. Understanding datasets: Understanding a dataset means checking rows, columns, and data types first - a basic manual look, unlike profiling (2e) which automates this - as you'd say: 'Before analyzing anything, I check the dataset's shape, columns, and data types to know what I'm working with.'

1c. Identifying patterns: Identifying patterns means finding repeating trends that show what's 'normal' - the opposite of anomalies, which break that pattern - as you'd say: 'I look for patterns and correlations to understand relationships between features.'

1d. Detecting anomalies: Detecting anomalies means finding unusual/wrong values using IQR or Z-score - different from a rare-but-valid value, since anomalies are usually errors - as you'd say: 'I use IQR or Z-score to detect outliers or errors before modeling.'

1e. Data storytelling: Data storytelling means turning findings into a clear explanation stakeholders can act on - different from a chart alone, which only shows data - as you'd say: 'Data storytelling means translating numbers into a clear insight stakeholders can act on.'

Recap: Topic 2: Understand Descriptive Analysis

Descriptive analysis means summarizing data using numbers like average, spread, and shape to describe it briefly.

2a. Summary statistics: Summary statistics are a quick set of numbers describing a column at a glance - broader than central tendency, which is just the 'center' part - as you'd say: 'Summary statistics give a quick numerical overview using mean, min, max, and standard deviation.'

2b. Distribution analysis: Distribution analysis looks at the shape of spread (symmetric, skewed) - different from variability, which measures how much spread, not shape - as you'd say: 'Distribution analysis helps me see whether data is symmetric or skewed, affecting preprocessing choices.'

2c. Variability analysis: Variability analysis measures consistency using one number like std dev - different from distribution, which shows the whole shape - as you'd say: 'Variability analysis, using std dev or IQR, tells me how consistent or scattered the data is.'

2d. Central tendency analysis: Central tendency finds the single typical value (mean/median/mode) - narrower than summary stats, which also includes spread - as you'd say: 'I use mean, median, or mode depending on skew to represent the typical value.'

2e. Dataset profiling: Dataset profiling bundles every check into one report - different from 1b's manual look, since profiling is automated - as you'd say: 'Dataset profiling combines all checks into one comprehensive report before analysis.'

Recap: Topic 3: Understand Univariate Analysis

Univariate analysis means studying just one variable at a time, on its own, without comparing it to any other variable.

3a. Single-variable analysis: Studying one column in isolation is the foundational first step - different from bivariate/multivariate since no other variable is involved - as you'd say: 'I always analyze each variable individually before checking relationships between them.'

3b. Histograms: A histogram visually shows value distribution shape - different from a frequency table since it's graphical, not tabular - as you'd say: 'I use histograms to visually inspect the shape and skew of a numeric variable.'

3c. Frequency distributions: A frequency distribution counts occurrences per value/category - applies to both numeric and categorical data as counts, not shape - as you'd say: 'I check frequency distributions to understand class balance in categorical features.'

3d. Outlier detection: Outlier detection finds extreme values in one column using box plots or IQR - same concept as anomaly detection (1d) applied within univariate analysis - as you'd say: 'I use IQR and box plots to detect outliers within individual features.'

3e. Distribution interpretation: Interpretation converts histogram shape and stats into an actionable conclusion - drives next steps like transforms - as you'd say: 'After analyzing a distribution, I interpret it to decide if transformation or scaling is needed.'

Recap: Topic 4: Understand Bivariate Analysis

Bivariate analysis means studying the relationship between exactly two variables at a time.

4a. Feature relationships: Feature relationships means examining how two variables connect - the shift from univariate's single-variable focus - as you'd say: 'I check relationships between features to understand how variables might influence each other.'

4b. Correlation analysis: Correlation gives one number (-1 to +1) for linear relationship strength - narrower than feature relationships, which also covers categorical pairs - as you'd say: 'I use Pearson correlation to quantify strength and direction of linear relationships.'

4c. Scatter plots: A scatter plot visually shows the actual relationship shape - different from correlation, which gives only a number - as you'd say: 'I use scatter plots to visually confirm what a correlation coefficient suggests.'

4d. Comparative analysis: Comparative analysis compares a numeric variable across categorical groups - different from correlation/scatter which need two numeric variables - as you'd say: 'I use group-by comparisons and box plots to see how a numeric feature varies across categories.'

4e. Trend identification: Trend identification tracks directional patterns over an ordered sequence like time - different from a plain scatter, since it's sequence-based - as you'd say: 'I check for trends over time to see if a relationship is strengthening, weakening, or stable.'

Recap: Topic 5: Understand Multivariate Analysis

Multivariate analysis means studying the relationships between three or more variables at the same time.

5a. Multiple feature relationships: Studying how 3+ variables connect together rather than just pairs - different from bivariate's two-variable focus - as you'd say: 'I analyze combinations of features together since outcomes are rarely explained by one relationship.'

5b. Feature interaction analysis: Checks if one variable's effect changes depending on another - different from 5a since it's about effects influencing each other - as you'd say: 'I check for interactions to see if a feature's impact differs across subgroups.'

5c. Pattern discovery: Finds hidden groupings across multiple variables combined - different from 1c's simpler pattern-spotting - as you'd say: 'I look for multivariate patterns since some risk segments only appear when conditions combine.'

5d. Dimensional thinking: The mindset of treating each variable as an added dimension - conceptual, not a specific technique - as you'd say: 'I think of records as points in a multi-dimensional feature space, how models actually process data.'

5e. Business insights generation: Translates multivariate findings into an actionable recommendation - mirrors 1e/3e but for combined findings - as you'd say: 'I translate multivariate patterns into specific, actionable business recommendations.'

Recap: Topic 6: Understand Visualization Principles

Visualization principles are the rules and best practices for choosing and designing charts so they communicate data honestly and clearly.

6a. Choosing the right chart: Matching chart type to data and question - about selection, not design or accuracy - as you'd say: 'I pick chart types based on variable count and type - histograms, scatter plots, or bar charts.'

6b. Data storytelling (visual): Designing a chart to make its insight obvious via titles and highlights - about framing an already-chosen chart - as you'd say: 'I design charts with descriptive titles and highlighted key points so the insight is clear.'

6c. Visualization best practices: General hygiene rules - clear labels, scale, minimal clutter - apply to any chart regardless of narrative - as you'd say: 'I follow best practices like clear labels and reduced clutter to keep charts trustworthy.'

6d. Avoiding misleading visuals: Catching distortions like truncated axes - focuses on accuracy/honesty specifically - as you'd say: 'I check for distortions like non-zero axes to make sure a chart doesn't exaggerate differences.'

6e. Effective communication: The overarching goal of tailoring visuals to the audience - the audience-awareness layer over all other subtopics - as you'd say: 'I tailor the level of detail based on audience - technical teams get detail, executives get one takeaway.'

Recap: Topic 7: Understand ML-Oriented EDA

ML-Oriented EDA means doing exploratory analysis specifically to prepare data for machine learning.

7a. Target variable analysis: Studying the predicted column's type and balance - different from feature importance since it focuses only on the outcome - as you'd say: 'I check the target's balance and type first, since it determines model

type and metric.'

7b. Feature importance awareness: Early EDA-level estimate of predictive features using correlation/group comparisons - different from formal importance, which needs a trained model - as you'd say: 'I use correlation and group differences to get an early sense of predictive features.'

7c. Bias identification: Checks for representation gaps or historical unfairness in the data - about fairness, not correctness - as you'd say: 'I check for representation gaps and historical bias since a model can be accurate but unfair.'

7d. Data quality validation: Confirms the dataset is technically ready - no missing values, duplicates, or leakage - about correctness/usability, not fairness - as you'd say: 'I validate for missing values, duplicates, and leakage since these directly break model performance.'

7e. Preparing for model building: Consolidates every finding into a final readiness checklist - the capstone step of Topic 7 and Module 9 - as you'd say: 'I summarize target balance, feature signals, bias checks, and data quality into one readiness checklist.'